

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



LAB 3

[Code No: COMP 314]

(For partial fulfillment of Third Year/Second Semester in Computer Engineering)

Submitted by:

Roshan Sahani (Roll No. 42)

Submitted to

(Rajani Chulyadyo)

Department of Computer Science and Engineering

Submission Date: 11th June ,2024

Solving Knapsack problems using different algorithm design strategies.

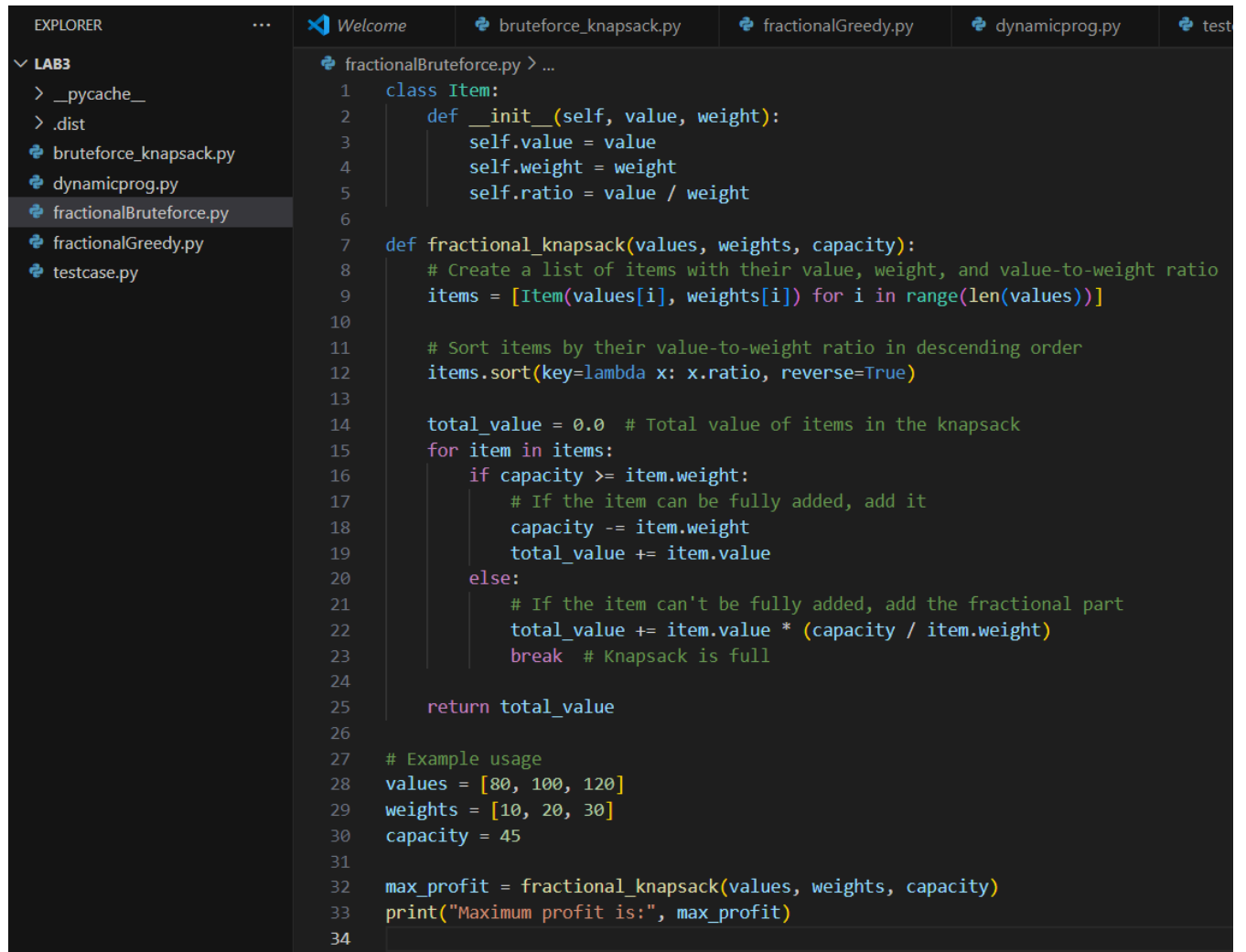
1. Brute-force method (Both fractional and 0/1 Knapsack)

• Fractional Brute-force Method

Pseudo Code

1. Verify that the length of values and weights are equal
2. Set n to the length of values
3. Create a class Item:
 - a) Initialize(value, weight):
 - i. Set self.value = value
 - ii. Set self.weight = weight
 - iii. Set self.ratio = value / weight
4. Define function fractional_knapsack(values, weights, capacity):
 - a) Create an empty list items
 - b) For i in the range of n:
 - i. Create an Item object with values[i] and weights[i]
 - ii. Append the Item object to items list
 - c. Sort items list by ratio in descending order
 - d. Initialize total_value = 0.0
 - e. For each item in items:
 - i. If capacity >= item.weight:
 1. Subtract item.weight from capacity
 2. Add item.value to total_value
 - ii. Else:
 1. Add item.value * (capacity / item.weight) to total_value
 2. Break the loop
 - f. Return total_value
5. Set max_profit to fractional_knapsack(values, weights, capacity)
6. Print "Maximum profit is:", max_profit

Source Code



```
fractionalBruteForce.py > ...
1  class Item:
2      def __init__(self, value, weight):
3          self.value = value
4          self.weight = weight
5          self.ratio = value / weight
6
7  def fractional_knapsack(values, weights, capacity):
8      # Create a list of items with their value, weight, and value-to-weight ratio
9      items = [Item(values[i], weights[i]) for i in range(len(values))]
10
11     # Sort items by their value-to-weight ratio in descending order
12     items.sort(key=lambda x: x.ratio, reverse=True)
13
14     total_value = 0.0 # Total value of items in the knapsack
15     for item in items:
16         if capacity >= item.weight:
17             # If the item can be fully added, add it
18             capacity -= item.weight
19             total_value += item.value
20         else:
21             # If the item can't be fully added, add the fractional part
22             total_value += item.value * (capacity / item.weight)
23             break # Knapsack is full
24
25     return total_value
26
27     # Example usage
28     values = [80, 100, 120]
29     weights = [10, 20, 30]
30     capacity = 45
31
32     max_profit = fractional_knapsack(values, weights, capacity)
33     print("Maximum profit is:", max_profit)
34
```

Output

```
27  # Example usage
28  values = [80, 100, 120]
29  weights = [10, 20, 30]
30  capacity = 45
31
32  max_profit = fractional_knapsack(values, weights, capacity)
33  print("Maximum profit is:", max_profit)
34  |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

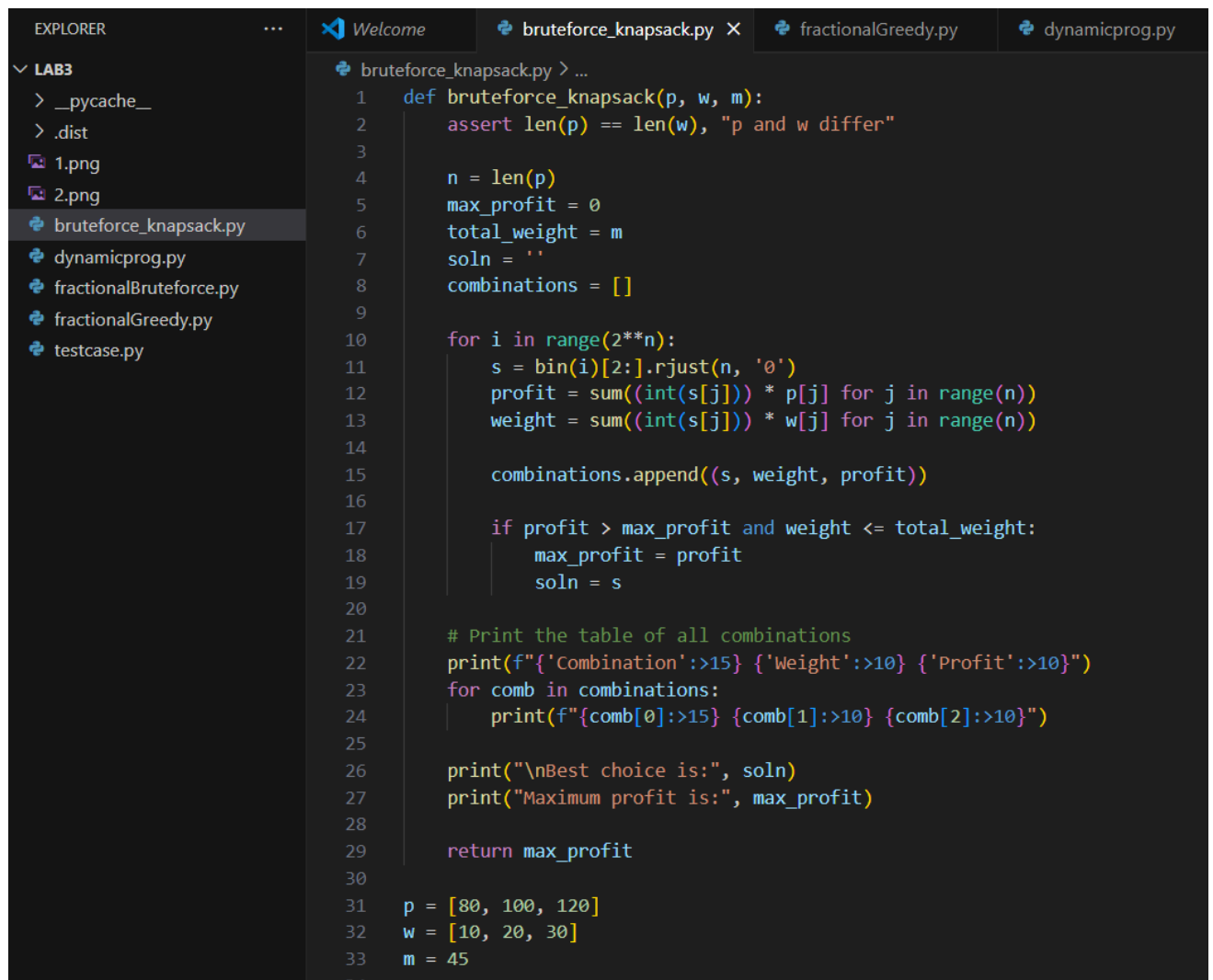
[Running] python -u "d:\6thSem\AlgoLab\Lab3\fractionalBruteforce.py"
Maximum profit is: 240.0

- **Solving 0/1 Knapsack using Brute-Force method**

Pseudo Code

1. Define function `bruteforce_knapsack(p, w, m)`:
 - a. Assert that length of `p` is equal to length of `w`, else raise an error "`p` and `w` differ"
 - b. Set `n` to the length of `p`
 - c. Initialize `max_profit` to 0
 - d. Initialize `total_weight` to `m`
 - e. Initialize `soln` to an empty string
 - f. Initialize an empty list `combinations`
 - g. For `i` in the range from 0 to $2^n - 1$:
 - i. Convert `i` to binary string `s`, padded with leading zeros to length `n`
 - ii. Initialize `profit` to the sum of `p[j]` for each `j` where `s[j]` is '1'
 - iii. Initialize `weight` to the sum of `w[j]` for each `j` where `s[j]` is '1'
 - iv. Append the tuple `(s, weight, profit)` to `combinations`
 - v. If `profit` is greater than `max_profit` and `weight` is less than or equal to `total_weight`:
 1. Set `max_profit` to `profit`
 2. Set `soln` to `s`
 - h. Print the table header "`Combination Weight Profit`"
 - i. For each combination in `combinations`:
 - i. Print the combination, its weight, and its profit formatted appropriately
 - j. Print "Best choice is:" followed by `soln`
 - k. Print "Maximum profit is:" followed by `max_profit`
 - l. Return `max_profit`
2. Call `bruteforce_knapsack(p, w, m)`

Source Code



```
1 def bruteforce_knapsack(p, w, m):
2     assert len(p) == len(w), "p and w differ"
3
4     n = len(p)
5     max_profit = 0
6     total_weight = m
7     soln = ''
8     combinations = []
9
10    for i in range(2**n):
11        s = bin(i)[2:].rjust(n, '0')
12        profit = sum((int(s[j])) * p[j] for j in range(n))
13        weight = sum((int(s[j])) * w[j] for j in range(n))
14
15        combinations.append((s, weight, profit))
16
17        if profit > max_profit and weight <= total_weight:
18            max_profit = profit
19            soln = s
20
21    # Print the table of all combinations
22    print(f"{'Combination':>15} {'Weight':>10} {'Profit':>10}")
23    for comb in combinations:
24        print(f"{'comb[0]':>15} {'comb[1]':>10} {'comb[2]':>10}")
25
26    print("\nBest choice is:", soln)
27    print("Maximum profit is:", max_profit)
28
29    return max_profit
30
31    p = [80, 100, 120]
32    w = [10, 20, 30]
33    m = 45
```

Output

```
31 p = [80, 100, 120]
32 w = [10, 20, 30]
33 m = 45
34
35 bruteforce_knapsack(p, w, m)
36
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

[Running] python -u "d:\6thSem\AlgoLab\Lab3\bruteforce_knapsack.py"

Combination	Weight	Profit
000	0	0
001	30	120
010	20	100
011	50	220
100	10	80
101	40	200
110	30	180
111	60	300

Best choice is: 101

Maximum profit is: 200

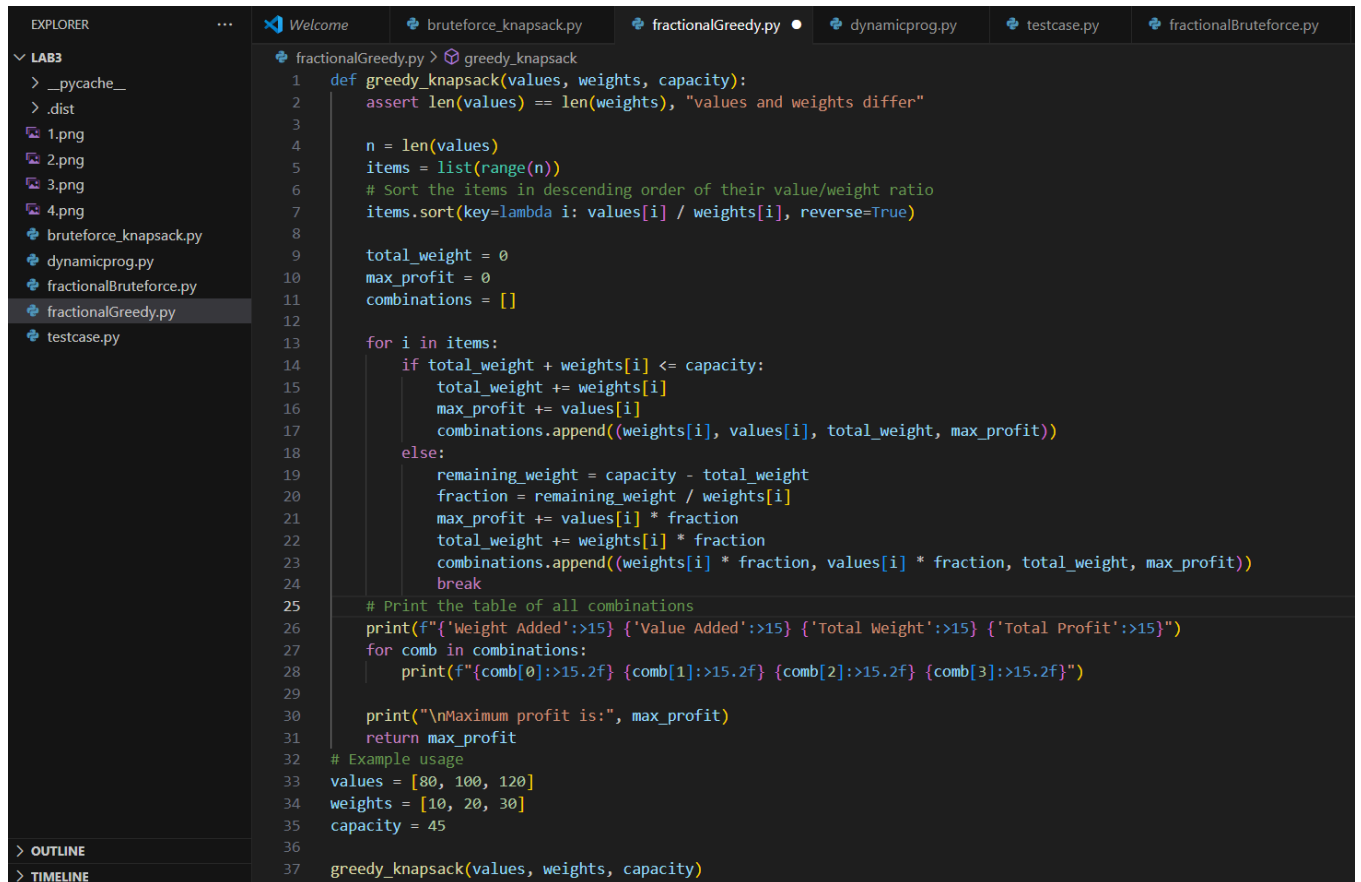
[Done] exited with code=0 in 0.198 seconds

2. Greedy method (Fractional Knapsack)

Pseudo Code

1. Define function greedy_knapsack(values, weights, capacity):
 - a. Assert that the length of values is equal to the length of weights, else raise an error "values and weights differ"
 - b. Set n to the length of values
 - c. Create a list items containing indices from 0 to n-1
 - d. Sort the items list in descending order based on the value-to-weight ratio of each item
 - e. Initialize total_weight to 0
 - f. Initialize max_profit to 0
 - g. Initialize an empty list combinations
 - h. For each i in items:
 - i. If total_weight + weights[i] <= capacity:
 1. Add weights[i] to total_weight
 2. Add values[i] to max_profit
 3. Append a tuple (weights[i], values[i], total_weight, max_profit) to combinations
 - ii. Else:
 1. Calculate remaining_weight as capacity - total_weight
 2. Calculate fraction as remaining_weight / weights[i]
 3. Add values[i] * fraction to max_profit
 4. Add weights[i] * fraction to total_weight
 5. Append a tuple (weights[i] * fraction, values[i] * fraction, total_weight, max_profit) to combinations
 6. Break the loop
- i. Print the table header "Weight Added Value Added Total Weight Total Profit"
- j. For each combination in combinations:
 - i. Print the weight added, value added, total weight, and total profit formatted appropriately
- k. Print "Maximum profit is:" followed by max_profit
- l. Return max_profit

Source Code



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'LAB3' with files: `__pycache__`, `.dist`, `1.png`, `2.png`, `3.png`, `4.png`, `bruteforce_knapsack.py`, `dynamicprog.py`, `fractionalBruteforce.py`, `fractionalGreedy.py` (selected), and `testcase.py`. The code editor shows the implementation of a greedy knapsack algorithm in Python. The code is as follows:

```
fractionalGreedy.py > greedy_knapsack
1 def greedy_knapsack(values, weights, capacity):
2     assert len(values) == len(weights), "values and weights differ"
3
4     n = len(values)
5     items = list(range(n))
6     # Sort the items in descending order of their value/weight ratio
7     items.sort(key=lambda i: values[i] / weights[i], reverse=True)
8
9     total_weight = 0
10    max_profit = 0
11    combinations = []
12
13    for i in items:
14        if total_weight + weights[i] <= capacity:
15            total_weight += weights[i]
16            max_profit += values[i]
17            combinations.append((weights[i], values[i], total_weight, max_profit))
18        else:
19            remaining_weight = capacity - total_weight
20            fraction = remaining_weight / weights[i]
21            max_profit += values[i] * fraction
22            total_weight += weights[i] * fraction
23            combinations.append((weights[i] * fraction, values[i] * fraction, total_weight, max_profit))
24            break
25
26    # Print the table of all combinations
27    print(f"{'Weight Added':>15} {'Value Added':>15} {'Total Weight':>15} {'Total Profit':>15}")
28    for comb in combinations:
29        print(f"{comb[0]:>15.2f} {comb[1]:>15.2f} {comb[2]:>15.2f} {comb[3]:>15.2f}")
30
31    print("\nMaximum profit is:", max_profit)
32    return max_profit
33
34    # Example usage
35    values = [80, 100, 120]
36    weights = [10, 20, 30]
37    capacity = 45
38
39    greedy_knapsack(values, weights, capacity)
```

Output

```
32 # Example usage
33 values = [80, 100, 120]
34 weights = [10, 20, 30]
35 capacity = 45
36
37 greedy_knapsack(values, weights, capacity)
38
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

[Running] python -u "d:\6thSem\AlgoLab\Lab3\fractionalGreedy.py"

Weight Added	Value Added	Total Weight	Total Profit
10.00	80.00	10.00	80.00
20.00	100.00	30.00	180.00
15.00	60.00	45.00	240.00

Maximum profit is: 240.0

[Done] exited with code=0 in 0.305 seconds

3. Dynamic programming (0/1 Knapsack)

Pseudo Code

1. Define function knapsack_dynamic(W, wt, val, n):
 - a. Create a 2D list K with dimensions (n+1) x (W+1), initialized with zeros
 - b. For i from 0 to n:
 - i. For w from 0 to W:
 1. If i is 0 or w is 0:
 - a. Set K[i][w] to 0
 2. Else if wt[i-1] <= w:
 - a. Set K[i][w] to the maximum of (val[i-1] + K[i-1][w-wt[i-1]]) and K[i-1][w]
 3. Else:
 - a. Set K[i][w] to K[i-1][w]
 - c. Return K[n][W]

Source Code

```
EXPLORER
...
LAB3
> __pycache__
> .dist
1.png
2.png
3.png
4.png
5.png
6.png
bruteforce_knapsack.py
dynamicprog.py
fractionalBruteforce.py
fractionalGreedy.py
testcase.py

dynamicprog.py > ...
1  def knapsack_dynamic(W, wt, val, n):
2      K = [[0 for x in range(W + 1)] for x in range(n + 1)]
3      for i in range(n + 1):
4          for w in range(W + 1):
5              if i == 0 or w == 0:
6                  K[i][w] = 0
7              elif wt[i-1] <= w:
8                  K[i][w] = max(val[i-1] + K[i-1][w-wt[i-1]], K[i-1][w])
9              else:
10                 K[i][w] = K[i-1][w]
11     return K[n][W]
12
13     val = [80, 100, 120]
14     wt = [10, 20, 30]
15     W = 45
16     n = len(val)
17     print("Maximum value in 0/1 knapsack using Dynamic method: ", knapsack_dynamic(W, wt, val, n))
18
```

Output

```
13     val = [80, 100, 120]
14     wt = [10, 20, 30]
15     W = 45
16     n = len(val)
17     print("Maximum value in 0/1 knapsack using Dynamic method: ", knapsack_dynamic(W, wt, val, n))
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

```
[Running] python -u "d:\6thSem\AlgoLab\Lab3\dynamicprog.py"
Maximum value in 0/1 knapsack using Dynamic method: 200

[Done] exited with code=0 in 0.196 seconds
```

4. Test cases for above programs

```
testcase.py > ...
1  import unittest
2  from bruteforce_knapsack import bruteforce_knapsack
3  from dynamicprog import knapsack_dynamic
4  from fractionalBruteforce import fractional_knapsack
5  from fractionalGreedy import greedy_knapsack
6  class KnapsackTest(unittest.TestCase):
7      def test_bruteforce_knapsack(self):
8          p = [280, 100, 120, 120]
9          w = [40, 10, 20, 24]
10         m = 60
11         expected_output = 400
12         result = bruteforce_knapsack(p, w, m)
13         self.assertEqual(result, expected_output)
14     def test_knapsack_dynamic(self):
15         val = [20, 5, 10, 40, 15, 25]
16         wt = [1, 2, 3, 8, 7, 4]
17         W = 10
18         n = len(val)
19         expected_output = 60
20         result = knapsack_dynamic(W, wt, val, n)
21         self.assertEqual(result, expected_output)
22     def test_fractional_knapsack(self):
23         values = [280, 100, 120, 120]
24         weights = [40, 10, 20, 24]
25         capacity = 60
26         expected_output = 440.0
27         result = fractional_knapsack(values, weights, capacity)
28         self.assertEqual(result, expected_output)
29     def test_greedy_knapsack(self):
30         values = [280, 100, 120, 120]
31         weights = [40, 10, 20, 24]
32         capacity = 60
33         expected_output = 440.0
34         result = greedy_knapsack(values, weights, capacity)
35         self.assertEqual(result, expected_output)
36 if __name__ == "__main__":
37     unittest.main()
```

Output

```
[Running] python -u "d:\6thSem\AlgoLab\Lab3\testcase.py"

  Combination    Weight    Profit
      000         0         0
      001        30        120
      010        20        100
      011        50        220
      100        10         80
      101        40        200
      110        30        180
      111        60        300

Best choice is: 101
Maximum profit is: 200
Maximum value in 0/1 knapsack using Dynamic method: 200
Maximum profit is: 240.0

  Weight Added    Value Added    Total Weight    Total Profit
      10.00         80.00         10.00         80.00
      20.00        100.00         30.00        180.00
      15.00         60.00         45.00        240.00

Maximum profit is: 240.0

  Combination    Weight    Profit
      0000         0         0
      0001        24        120
      0010        20        120
      0011        44        240
      0100        10        100
      0101        34        220
      0110        30        220
      0111        54        340
      1000        40        280
      1001        64        400
      1010        60        400
      1011        84        520
      1100        50        380
...      1101        74        500
      1110        70        500
      1111        94        620
```

Best choice is: 1010

Maximum profit is: 400

Weight Added	Value Added	Total Weight	Total Profit
10.00	100.00	10.00	100.00
40.00	280.00	50.00	380.00
10.00	60.00	60.00	440.00

Maximum profit is: 440.0

.

Ran 4 tests in 0.002s

OK

[Done] exited with code=0 in 0.415 seconds